

Igra nasprotnika: Pokrivanje košev

Navodila za uporabo

Dejan Jarc, Jure Zupančič

7. april 2026

1 Uvod

Za rešitev naloge uporabljamo dva programa:

- `bin_cover.java`, ki prebere vhodne datoteke, izračuna rešitev in ustvari izhodne datoteke,
- `validate_solutions.java`, ki preveri pravilnost ustvarjenih rešitev.

Program za preverjanje (oz. preverjevalnik) `validate_solutions.java` pričakuje mapo z vhodnimi datotekami in mapo z rešitvami, preverja pa datoteke oblike `input_*.txt`, za katere v mapi rešitev poišče ustrezne datoteke `output_*.txt`. Nato preveri, ali so indeksi veljavni, ali se kak predmet uporabi večkrat, ali je vsak prijavljen koš res pokrit in ali se dejansko število košev ujema s številom v prvi vrstici rešitve.

2 Prevajanje programov

Pred zagonom je treba oba programa prevesti. V ukazni vrstici se postavimo v mapo, kjer se nahajajo datoteke `bin_cover.java` in `validate_solutions.java`, nato izvedemo:

```
javac bin_cover.java
javac validate_solutions.java
```

Možno je tudi prevesti oba hkrati:

```
javac bin_cover.java validate_solutions.java
```

3 Struktura vhodnih podatkov

Vhodne datoteke morajo biti shranjene v posebni mapi (npr. `./inputs`). Vsaka vhodna datoteka vsebuje:

- v prvi vrstici število predmetov,
- v naslednjih vrsticah velikosti posameznih predmetov.

Primer vhodne datoteke:

5
0.45
0.30
0.60
0.20
0.50

Preverjevalnik zahteva, da morajo biti vhodne datoteke poimenovane v formatu *input_X.txt*, kjer *X* predstavlja zaporedno številko vhodne datoteke. Primeri:

input_0.txt
input_1.txt
input_2.txt
...

4 Zagon programa za pokrivanje košev

Program `bin_cover` za argumenta prejme:

- `input_folder_path`: pot do mape z vhodnimi datotekami,
- `output_folder_path`: pot do mape, kamor želimo shraniti izhodne datoteke.

Zagon programa:

```
java bin_cover <input_folder_path> <output_folder_path>
```

Primer:

```
java bin_cover ./inputs ./outputs
```

Če izhodna mapa še ne obstaja, jo program ustvari na začetku. Program prebere vse vhodne datoteke iz podane vhodne mape, jih obdela po vrsti in za vsako ustvari pripadajočo izhodno datoteko.

Izhodna datoteka ima enako številko kot vhodna, le da se ime spremeni iz `input_*` v `output_*`.

5 Oblika izhodnih podatkov

Izhodni podatki se shranijo v ločene izhodne datoteke, ki so ob koncu izvajanja programa shranjene v ločeno mapo (npr. `./outputs`). Vsaka izhodna datoteka vsebuje:

- v prvi vrstici število pokritih košev,
- v vsaki naslednji vrstici indekse predmetov, ki pripadajo enemu pokritemu košu.

Primer izhodne datoteke:

```
2  
1 3  
2 4 5
```

To pomeni, da sta bila ustvarjena dva pokrita koša:

- prvi vsebuje predmeta 1 in 3,
- drugi vsebuje predmete 2, 4 in 5.

6 Zagon preverjevalnika

Preverjevalnik se zažene z naslednjim ukazom:

```
java validate_solutions <input_folder> <solutions_folder>
```

Primer:

```
java validate_solutions ./inputs ./outputs
```

Preverjevalnik za vsako vhodno datoteko poišče rešitev z ustreznim imenom. Če je vhodna datoteka na primer:

input_7.txt

mora biti pripadajoča rešitev v datoteki:

output_7.txt

7 Kaj preverjevalnik preverja

Preverjevalnik za vsako rešitev preveri naslednje:

1. ali rešitev sploh obstaja,
2. ali je prva vrstica pravilno podano število košev,
3. ali so vsi uporabljeni indeksi predmetov znotraj dovoljenega območja,
4. ali se isti predmet pojavi največ enkrat,
5. ali je vsota velikosti v vsakem košu vsaj 1,
6. ali se prijavljeno število košev ujema z dejanskim številom vrstic v rešitvi.

Pri preverjanju pokritosti uporablja majhno toleranco zaradi računanja z realnimi števili. Koš je zato sprejet, če je njegova vsota dovolj blizu vrednosti 1.

8 Možni izpisi preverjevalnika

Preverjevalnik lahko za posamezno datoteko izpiše:

1. VALID (OK) – rešitev je pravilna,
2. NO SOLUTION FILE – pripadajoča izhodna datoteka ne obstaja,
3. ERROR -> ... – v rešitvi je bila najdena napaka.

Možni primeri napak so:

- neveljaven indeks predmeta,
- večkratna uporaba istega predmeta,
- koš ni pokrit,
- napačno prijavljeno število košev,
- prazna vhodna ali izhodna datoteka.

9 Primer celotnega postopka uporabe

1. pripravimo mapo z vhodnimi datotekami,
2. prevedemo oba programa (`bin_cover` in `validate_solutions`),
3. zaženemo `bin_cover`,
4. nato zaženemo `validate_solutions` in pregledamo izpis validatorja ter
5. po potrebi popravimo program.

Za ta postopek moramo v ukazni vrstici zagnati sledeče ukaze:

```
javac bin_cover.java validate_solutions.java
java bin_cover ./inputs ./outputs
java validate_solutions ./inputs ./outputs
```